Multidimensional Scaling

Jyotishka Datta

2020/01/24 (updated: 2025-02-23)

Idea of MDS

- Represent high-dimensional point cloud in few (usually 2) dimensions keeping distances between points similar.
- Useful tool in visualizing any big data-set, specially for clustering purposes.
- Is a popular exploratory tool. Used before any inferential procedure.

Goal of MDS

- Given pairwise dissimilarities, reconstruct a map that preserves distances.
- From any dissimilarity (no need to be a metric)
- Reconstructed map has coordinates $\mathbf{x}_i = (x_{i1}, x_{i2})$ and the natural distance $\|x_i - x_j\|^2$.
- MDS is a family of different algorithms, each designed to arrive at optimal low-dimensional configuration (p = 2 or 3)
- Includes: Classical MDS, Metric MDS and Non-metric MDS.

Examples first - Classical MDS

- Problem: Given Euclidean Distance between points, recover the position of the points.
  - Example: Road distance between 21 european cities

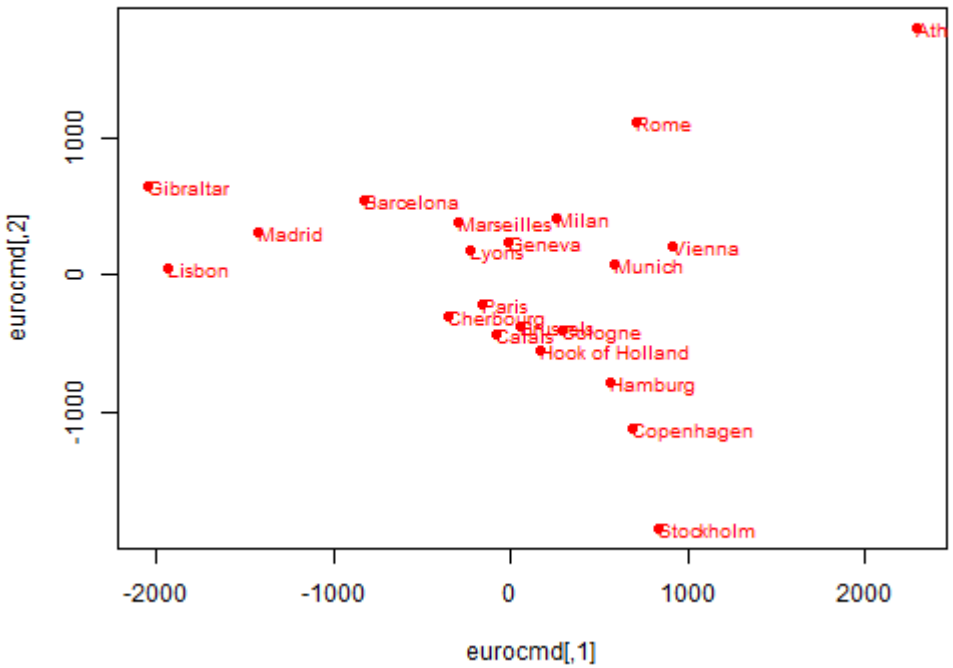
```
library(datasets); class(eurodist)
```

```
## [1] "dist"
```

```
                 Athens Barcelona Brussels Calais Cherbourg Cologne
Barcelona          3313
Brussels           2963      1318
Calais             3175      1326      204
Cherbourg          3339      1294      583    460
Cologne            2762      1498      206    409       785
```

Output of MDS

```
eurocmd <- cmdscale(eurodist)
plot(eurocmd, type = "p", pch = 16, col = "red")
text(eurocmd, rownames(eurocmd), adj = c(0,0.5), col = "red", cex = 0
```

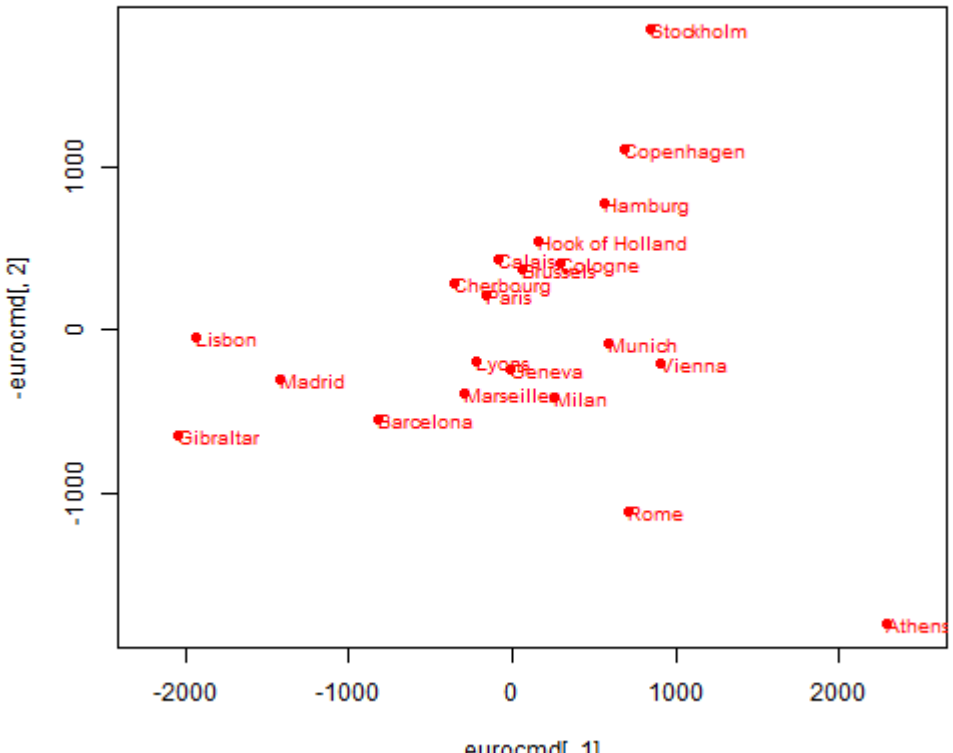


Do we recover?

- Can identify points up to shift, reflection and rotation.

Flip Axes

```
plot(eurocmd[,1], -eurocmd[,2], type = "p", asp = 1,pch = 16, col = '
text(eurocmd[,1], -eurocmd[,2], rownames(eurocmd),, adj = c(0,0.5), c
```



Another Example

- Air pollution in US Cities

```r
data("USairpollution", package = "HSAUR2")
summary(USairpollution)
```

```
##       SO2              temp            manu            popul
##  Min.   :  8.00   Min.   :43.50   Min.   :  35.0   Min.   :  71.0
##  1st Qu.: 13.00   1st Qu.:50.60   1st Qu.: 181.0   1st Qu.: 299.0
##  Median : 26.00   Median :54.60   Median : 347.0   Median : 515.0
##  Mean   : 30.05   Mean   :55.76   Mean   : 463.1   Mean   : 608.6
##  3rd Qu.: 35.00   3rd Qu.:59.30   3rd Qu.: 462.0   3rd Qu.: 717.0
##  Max.   :110.00   Max.   :75.50   Max.   :3344.0   Max.   :3369.0
##       wind            precip         predays
##  Min.   : 6.000   Min.   : 7.05   Min.   : 36.0
##  1st Qu.: 8.700   1st Qu.:30.96   1st Qu.:103.0
##  Median : 9.300   Median :38.74   Median :115.0
##  Mean   : 9.444   Mean   :36.77   Mean   :113.9
##  3rd Qu.:10.600   3rd Qu.:43.11   3rd Qu.:128.0
##  Max.   :12.700   Max.   :59.80   Max.   :166.0
```

- Some variables have larger range - need to standardise.

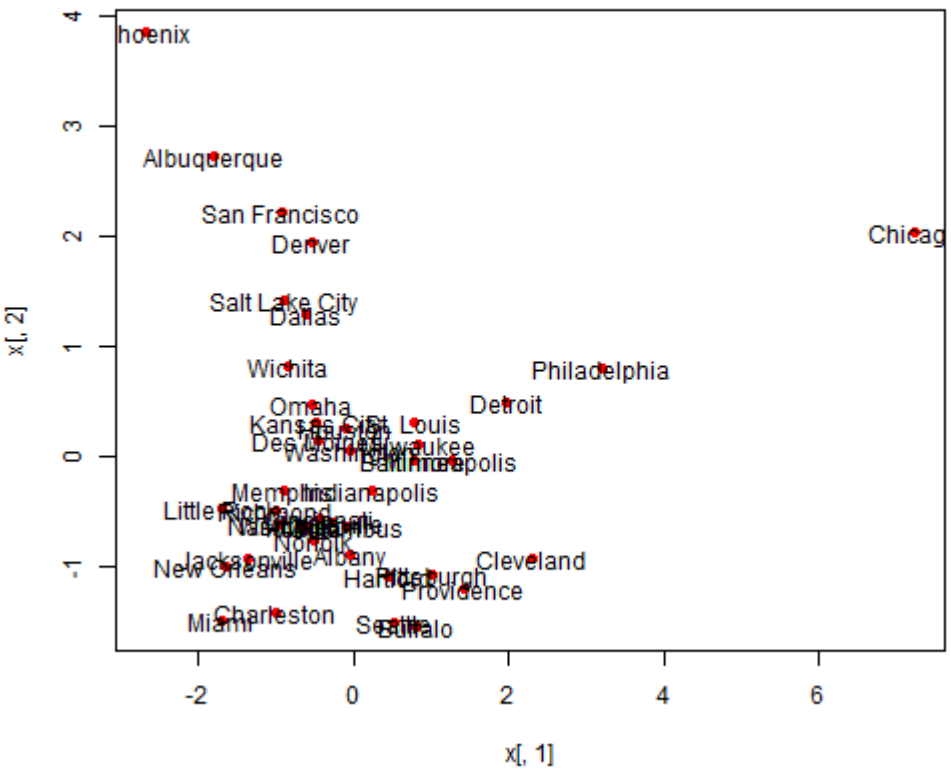
Try MDS at 2-D

```
dat <- USairpollution # less typing
xs <- scale(dat)
pol.mds <- cmdscale(dist(xs), k = 2, eig = TRUE)
head(pol.mds$points)
```

```
##                    [,1]        [,2]
## Albany      -0.03386467 -0.89884070
## Albuquerque -1.79452779  2.72940259
## Atlanta     -0.61099829 -0.63025982
## Baltimore    0.79088751 -0.03447144
## Buffalo      0.81715797 -1.54968000
## Charleston  -0.99951059 -1.41323028
```

- You can think of these as coordinates in some **latent** space.

Distance $\neq$ Physical Distance

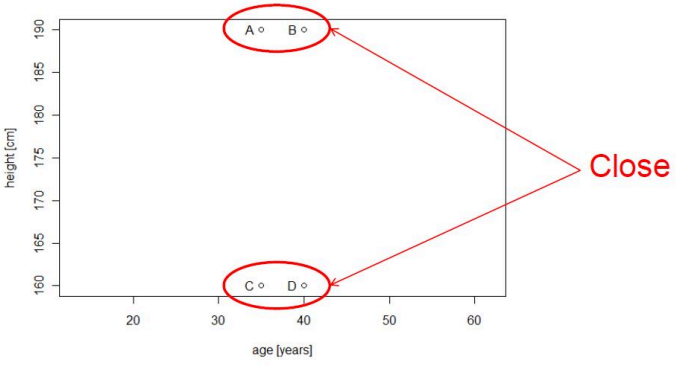


- SLC and Dallas have similar pollution profile. Chicago is an outlier.

To scale or not to scale

4 persons data on age and height.

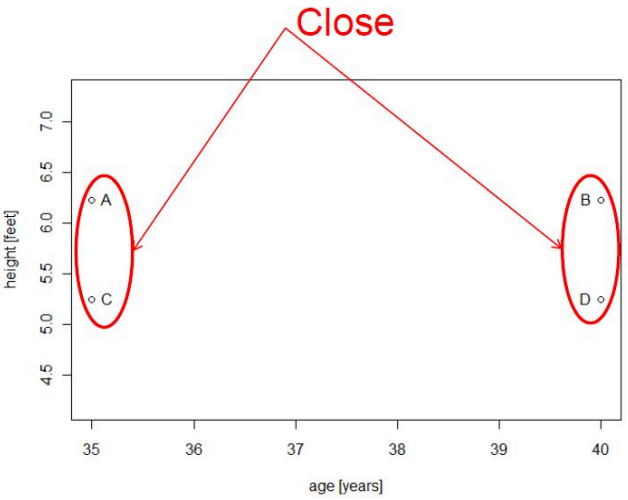
Person	Age [years]	Height [cm]
A	35	190
B	40	190
C	35	160
D	40	160



To scale or not to scale

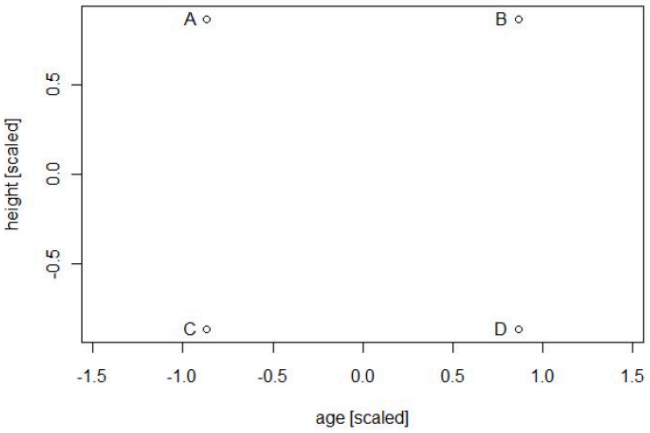
Person	Age [years]	Height [ft]
A	35	6.232
B	40	6.232
C	35	5.248
D	40	5.248

Convert cm to feet: different subgroups emerge.



To scale or not to scale

Person	Age [years]	Height [cm]
A	-0.87	0.87
B	0.87	0.87
C	-0.87	-0.87
D	0.87	-0.87

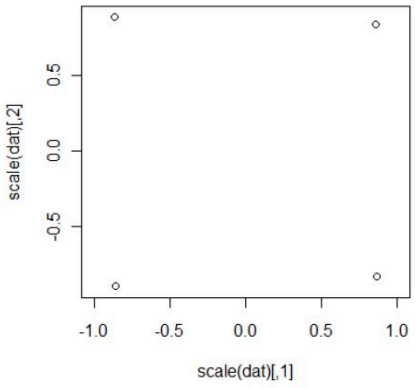


Now all data-points are scaled : no subgroups.

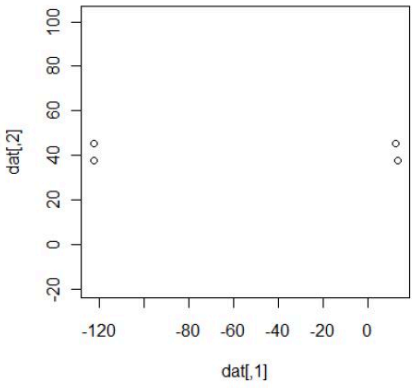
Context is important

Which of the two representations make sense?

Object	X1	X2
A	13.3	38
B	12.4	45.4
C	-122.7	45.6
D	-122.4	37.7



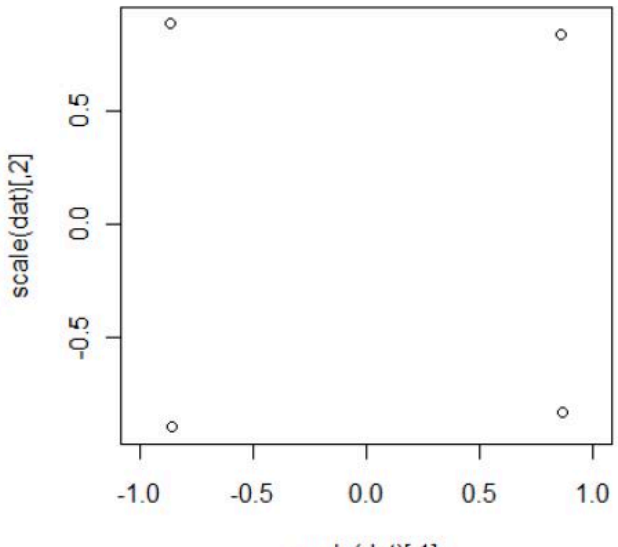
OR



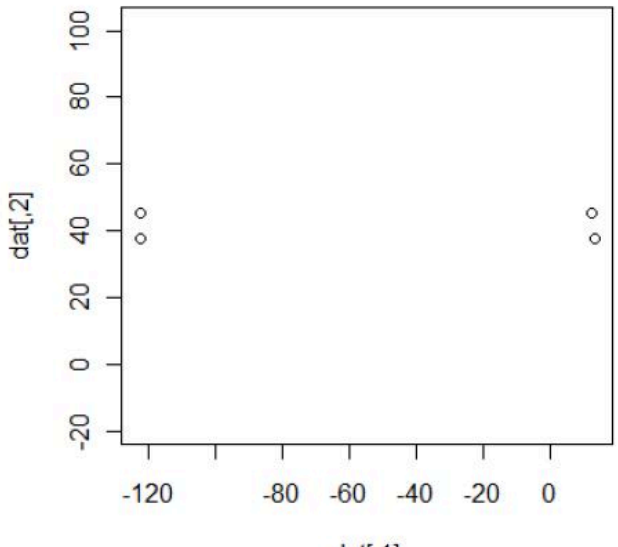
Context is important

You need knowledge of the context. Scaling is not a good idea always.

Object	Longitude	Latitude
Palermo	13.3	38
Venice	12.4	45.4
Portland	-122.7	45.6
San Francisco	-122.4	37.7



OR



Thoery of MDS

- Given a dissimilarity matrix $D = (d_{ij})$, MDS seeks to find $x_1, \ldots, x_n \in \mathbb{R}^p$, such that:

$$d_{ij} \approx \|x_i - x_j\|^2 \text{ as close as possible}$$

- Often times, for some large p, there exists a configuration $x_1, \ldots, x_n$ with exact distance match $d_{ij} = \|x_i - x_j\|^2$.
  - In such a case d is called a Euclidean distance.
  - There are, however, cases where the dissimilarity is distance, but there exists no configuration in any p with perfect match.
  - Such a distance is called non-Euclidean distance.

Classical MDS: Thoery (continued)

- Suppose for now we have Euclidean distance matrix $D = (d_{ij})$
- We want to find $(x_1, \ldots, x_n)$ such that $d_{ij} = dist(x_i, x_j)$.
- Not unique solution: $x^* = x + c$.
- Assume the observations are centered.

Thoery - Step 1

- Find inner product matrix $B = XX^T$ instead of X.
- Connect to distance: $d_{ij}^2 = b_{ii} + b_{jj} - 2b_{ij}$.
- Center points to avoid shift invariance.
- Invert relationship:

$$b_{ij} = -1/2(d_{ij}^2 - d_{i.}^2 - d_{.j}^2 + d_{..}^2)$$

- "Doubly centered" distance.

Theory - Step 2

- Since $B = XX^T$, we need the square-root of B.
- B is symmetric and positive definite.
- Can be diagonalised: $B = V\Lambda V^T$.
- Λ is diagonal matrix with eigenvalues $\lambda_1 > \lambda_2 > \cdots > \lambda_n$
- Some eignevalues are zero, drop them. $B = V_1\Lambda_1 V_1^T$.
- Take "square-root" $X = V_1\Lambda_1^{1/2}$.

Classical MDS

- Want a 2-D plot.
- Keep only largest 2 eignevectors and eignevalues. - The resulting X will be the low-dimensional representation we were looking for.
- Same for 3-D.

Classical MDS: Low-dim representation

- Goodness of fit (GOF) if we reduce to m dimensions:

$$\mathrm{GOF}_m = \frac{\sum_{i=1}^{m} \lambda_i}{\sum_{i=1}^{n} \lambda_i}$$

- We can choose m dimensional representation if $\mathrm{GOF}_m \geq 0.8$.

Classical MDS: Pros and Cons

Pros:

- Optimal for euclidean input data
- Still optimal, if B has non-negative eigenvalues (pos. semidefinite)
- Very fast

Cons:

- No guarantees if B has negative eigenvalues.

However, in practice, it is still used then. New measures for Goodness of fit:

$$\mathrm{GOF}_m = \frac{\sum_{i=1}^m |\lambda_i|}{\sum_{i=1}^n |\lambda_i|} \text{ or } \mathrm{GOF}_m = \frac{\sum_{i=1}^m \|\lambda_i\|_\infty}{\sum_{i=1}^n \|\lambda_i\|_\infty}$$

- These are used in the `R` function `cmdscale`.

Non-metric MDS: Idea

- Sometimes, there is no strict metric on original point
- Example: how well do you like something? (1 - not at all, 10 - very much)

Subject	Teacher-1	Teacher-2	Teacher-3
S1	1	5	10
s2	2	6	9
S3	1	7	8

- Teacher 1 < Teacher 2 < Teacher 3

Non-metric MDS

- Absolute values are not that meaningful
- Ranking is important
- Non-metric MDS finds a low-dimensional representation, which respects the ranking of distances.

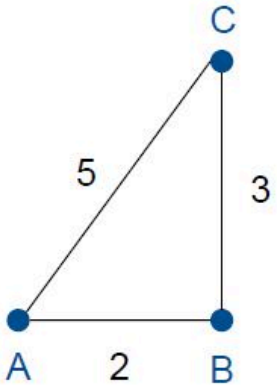
Non-metric MDS : Theory

- δ_{ij} is the true dissimilarity: d_{ij} is the distance of representation.
- Minimize Stress:

$$S = \frac{\sum_{i<j}(\theta(\delta_{ij}) - d_{ij})^2}{\sum_{i<j} d_{ij}^2}, \ \theta \text{ is an increasing function.}$$

- Optimize over "both" positions of points and θ.
- $\hat{d_{ij}} = \theta(\delta_{ij})$ is called "disparity".
- Solved numerically (isotonic regression); Classical MDS as starting value;
- very time consuming

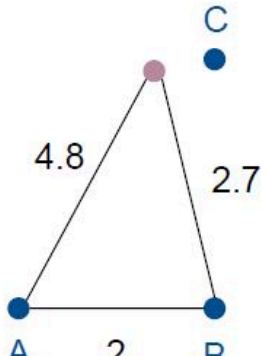
Example for Intuition



Compute best representation

STRESS = 19.7

$d_{AB} < d_{BC} < d_{AC}$



Compute best representation

STRESS = 20.1

Non-metric MDS: Pros and Cons

- Fulfills a clear objective without many assumptions (minimize STRESS)
- Results don’t change with rescaling or monotonic variable transformation
- Works even if you only have rank information

- Slow in large problems
- Usually only local (not global) optimum found
- Only gets ranks of distances right

Voting Data

- Look at the voting data.
- Are Democrats close to each other, republicans?

```
data("voting", package = "HSAUR")
require("MASS")
```

```
## Loading required package: MASS
```

```
head(voting[,1:4], n = 4)
```

```
##              Hunt(R) Sandman(R) Howard(D) Thompson(D)
## Hunt(R)            0          8        15          15
## Sandman(R)         8          0        17          12
## Howard(D)         15         17         0           9
## Thompson(D)       15         12         9           0
```

Metric MDS

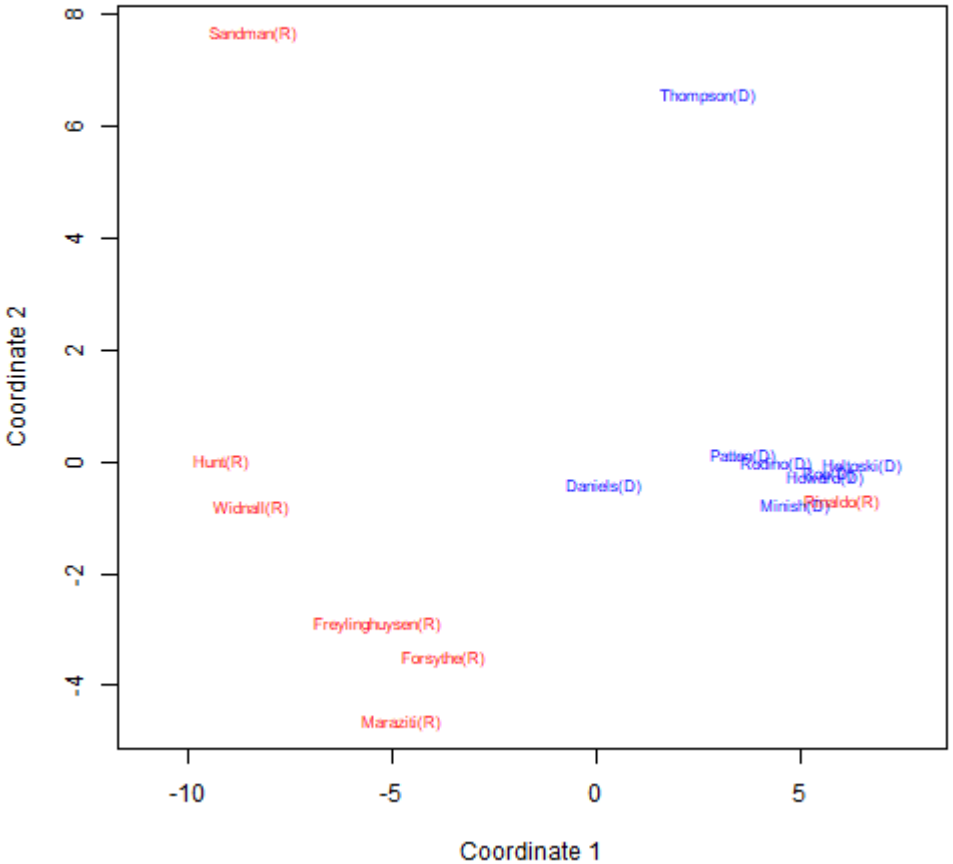
- For metric MDS, use function `cmdscale`.

```
voting.mds <- cmdscale(voting, eig=TRUE)
names(voting.mds)
```

```
## [1] "points" "eig"    "x"      "ac"     "GOF"
```

Plot

- What do the coordinates represent?

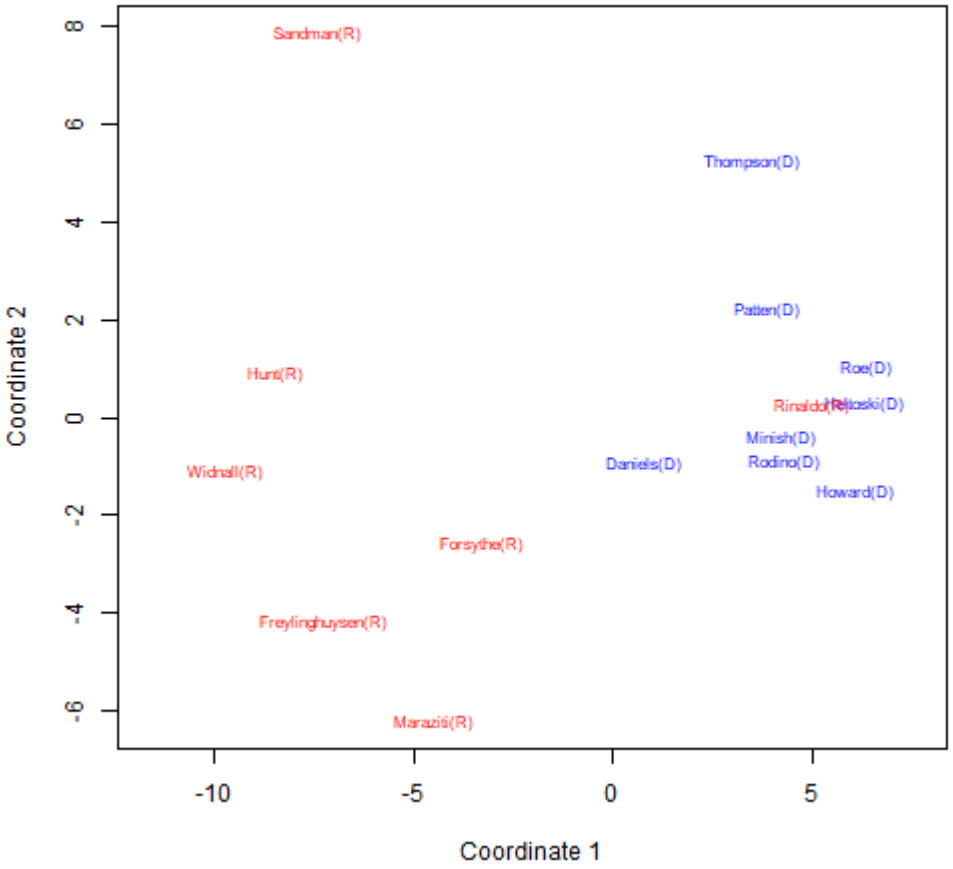


Solution - non-metric

```r
voting_mds <- isoMDS(voting)
```

```
## initial  value 15.268246
## iter   5 value 10.264075
## final  value 9.879047
## converged
```

Plot the non-metric MDS



Use `ggrepel`

- Exercise: how would you add colour to the text labels?

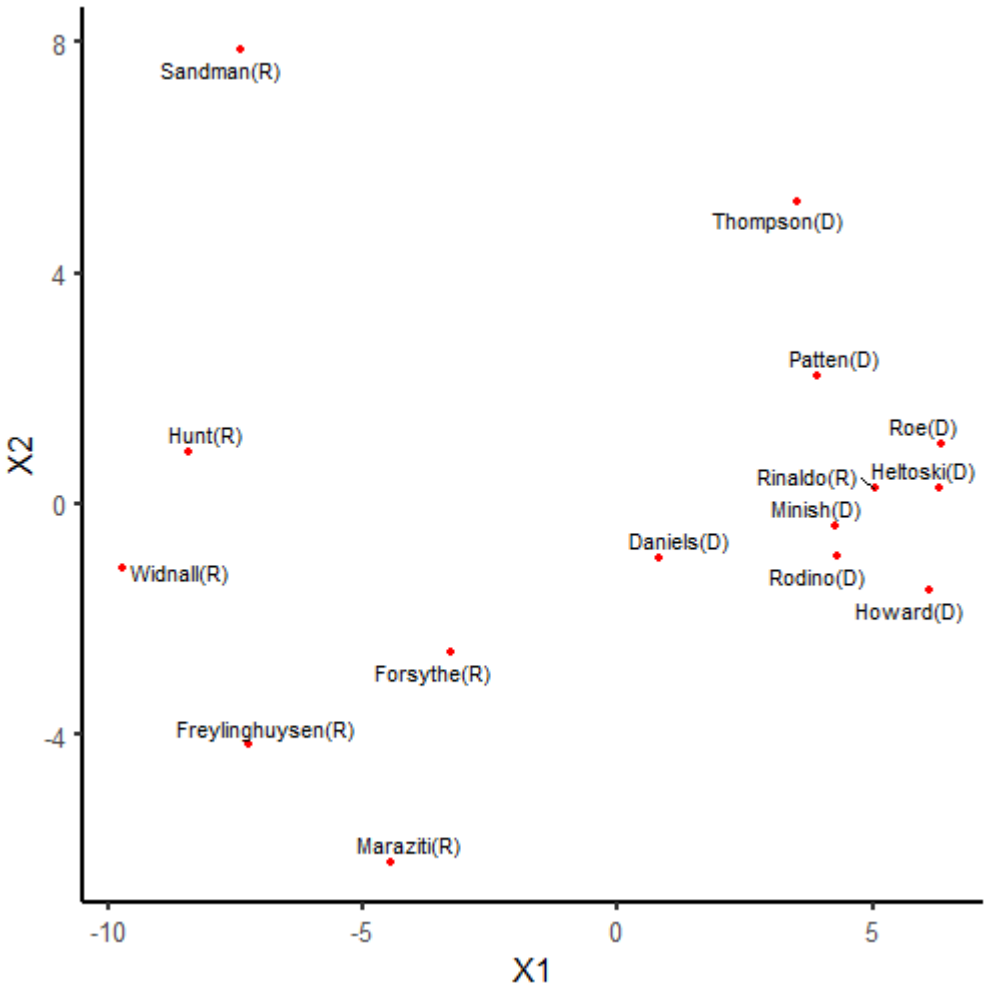
```
library(ggplot2)
library(ggrepel)

demind <- grep("(D)",colnames(voting))
col <- rep("red",15)
col[demind] <- "blue"

voting.data <- rbind(data.frame(voting_mds$points))

set.seed(42)
ggplot(voting.data) +
  geom_point(aes(X1,X2)) +
  geom_text_repel(aes(X1,X2,
                      label = rownames(voting.data))) +
  theme_classic(base_size = 16)
```

Use `ggrepel` fo better plots



Main message

Classical MDS:

- Finds low-dim projection that respects distances - Optimal for euclidean distances
- No clear guarantees for other distances
- fast

Non-metric MDS:

- Squeezes data points on table
- respects only rankings of distances
- (locally) solves clear objective
- slow

Random forest for MDS

Random Forest and MDS

- In absence of a distance measure, we can use random forest to get a proximity measure.
- Proximity measure: The (i, j) element of the proximity matrix produced by `randomForest` is the fraction of trees in which elements i and j fall in the same terminal node.
- The intuition is that **similar** observations should be in the same terminal nodes more often than dissimilar ones.
- We can use the proximity measure to identify structure.

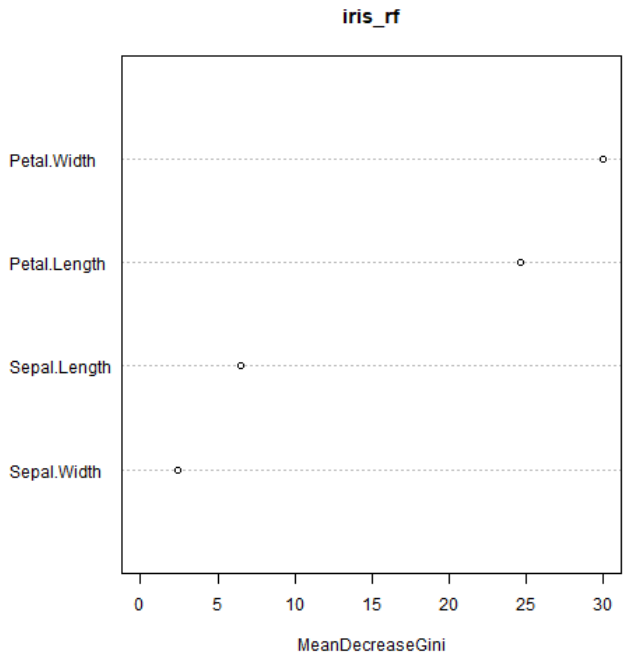
Random forest on `iris` data

```
library(randomForest)
ind <- sample(2,nrow(iris),replace=TRUE,prob=c(0.7,0.3))
trainData <- iris[ind==1,]
testData <- iris[ind==2,]
iris_rf <- randomForest(Species~.,data=trainData,ntree=100,proximity=
table(predict(iris_rf),trainData$Species)
```

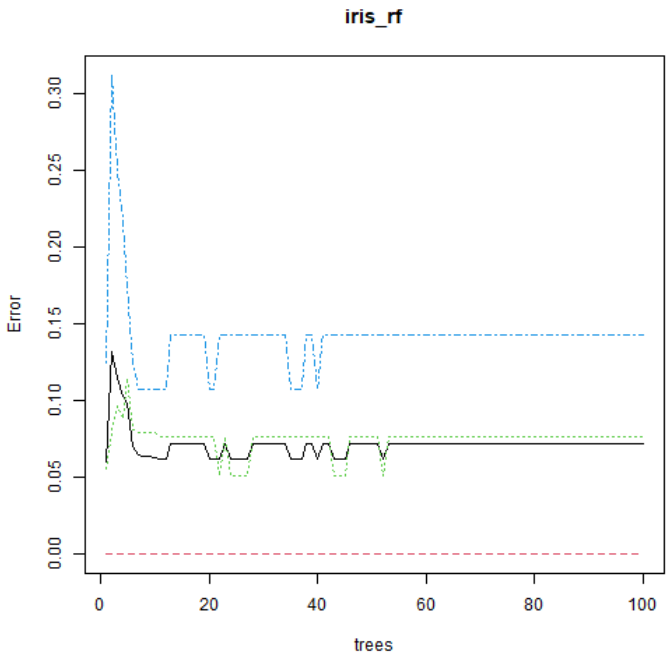
```
##
##              setosa versicolor virginica
##   setosa         30          0         0
##   versicolor      0         36         4
##   virginica       0          3        24
```

Random forest on `iris` data

```
varImpPlot(iris_rf)
```




```
plot(iris_rf)
```



Prediction on new data

```
irisPred<-predict(iris_rf,newdata=testData)
table(irisPred, testData$Species)
```

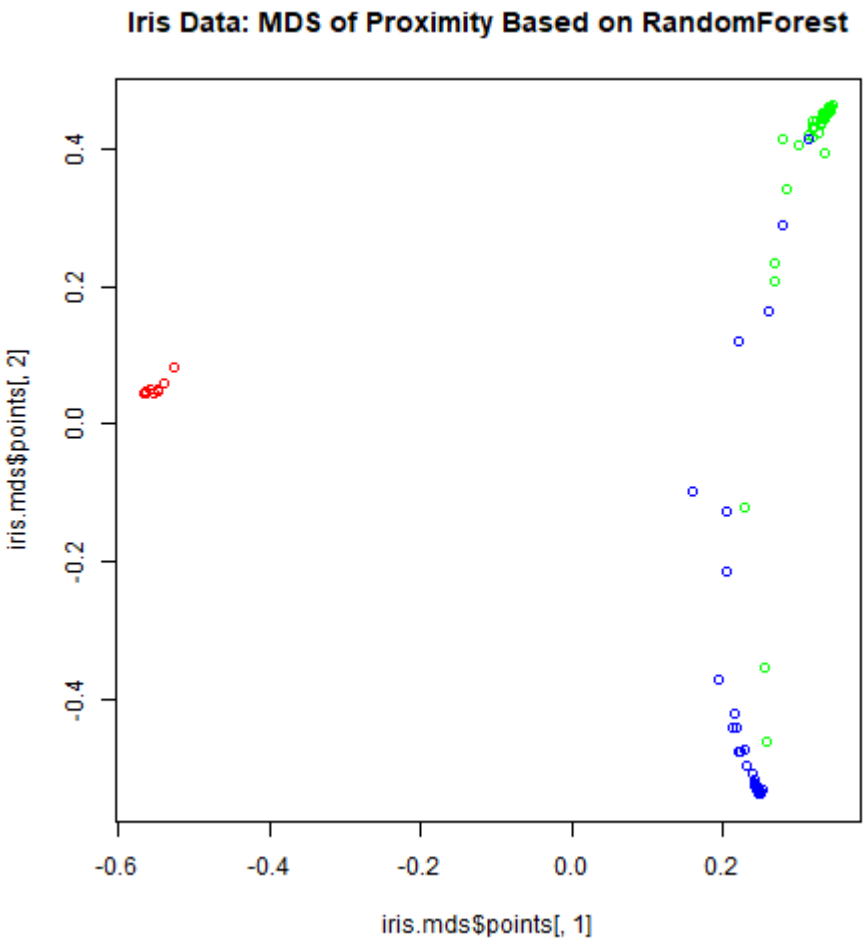
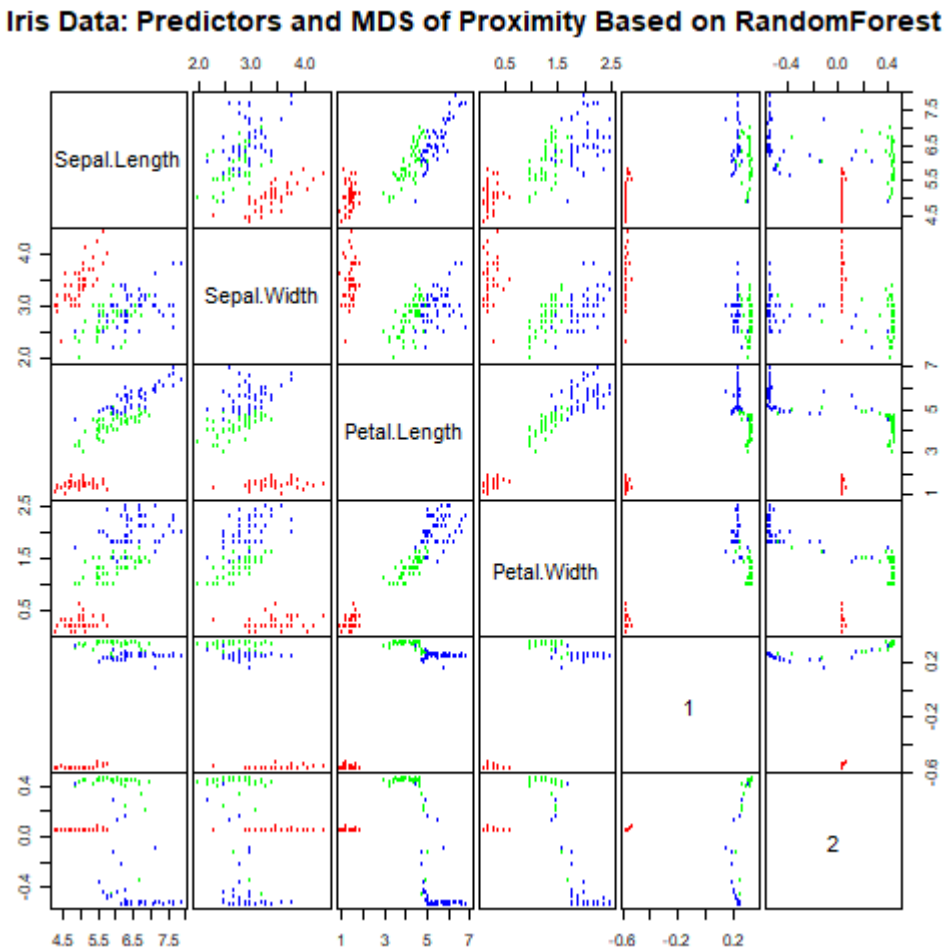
```
##
## irisPred     setosa versicolor virginica
##   setosa         20          0         0
##   versicolor      0         11         1
##   virginica       0          0        21
```

MDS on the full Iris Data

```
set.seed(71)
iris.rf <- randomForest(Species ~ ., data=iris, importance=TRUE,
                        proximity=TRUE)
## Do MDS on 1 - proximity:
iris.mds <- cmdscale(1 - iris.rf$proximity, eig=TRUE)
## Look at variable importance:
round(importance(iris.rf), 2)
```

```
##              setosa versicolor virginica MeanDecreaseAccuracy MeanDecrease
## Sepal.Length   5.88       5.87      9.21                10.62
## Sepal.Width    5.23       0.31      4.71                 4.94
## Petal.Length  21.60      31.41     27.71                32.39            4
## Petal.Width   22.96      33.74     32.07                33.85            4
```

Plot the MDS coordinates



Plot the MDS coordinates with the original variables